KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

GIT & LINUX Command Assistant:
A Retrieval-Augmented Generation (RAG) System

Project ID: PRJ020

Team ID: T020

Team GitHub Repository:
https://github.com/Satish7744/GEN_AI_MASTERY_PROJECT

Student Name	Registration Number	Individual GitHub Link
M Bharath Kumar Reddy	99230040681	https://github.com/bharathmule05/Gen-AI-tasks
Mangali Bhaskar	99230040657	https://github.com/bhaskar-mb/GenAi
Mareedu Indra Dinesh	99230040370	https://github.com/dineshmareedu15-spec/GEN_AI_MASTERY_PRACTICE
Vummadichetty Jaya Varshini	99230040812	https://github.com/varshini04-ux/GEN_AI_Practice/tree/main
Pasupuleti Satish	99230040716	https://github.com/Satish7744/GEN_AI_Practice

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

1. Introduction

Large Language Models (LLMs) are strong at language, but they can "hallucinate" and they have no access to specialised or private knowledge. Retrieval-Augmented Generation (RAG) reduces this problem by fetching relevant text from an external knowledge base before the model writes its answer.

This project implements a RAG system that answers questions about Git and Linux commands: how to use them, what their flags mean, how to fix common errors, and which operations are risky. It includes a Streamlit chat interface with conversation memory and a Troubleshoot mode for pasted terminal errors.

2. Problem Statement & Objectives

Problem: Beginners and students lose time moving between man pages, forum threads and tutorials to find the right Git or Linux command. A general chatbot may invent flags or suggest unsafe commands, and a keyword search cannot match a plain-language question such as "undo my last commit" to the correct command.

Objectives:

- To build an assistant that answers Git and Linux questions based on a curated knowledge base.
- To use a vector database for semantic search, so plain-language questions find the right entries.
- To reduce hallucinations by restricting the LLM to the retrieved text, and to refuse when the knowledge base has no answer.
- To support multi-turn chat and a Troubleshoot mode that returns a structured diagnosis of an error.

3. System Architecture (RAG Pipeline)

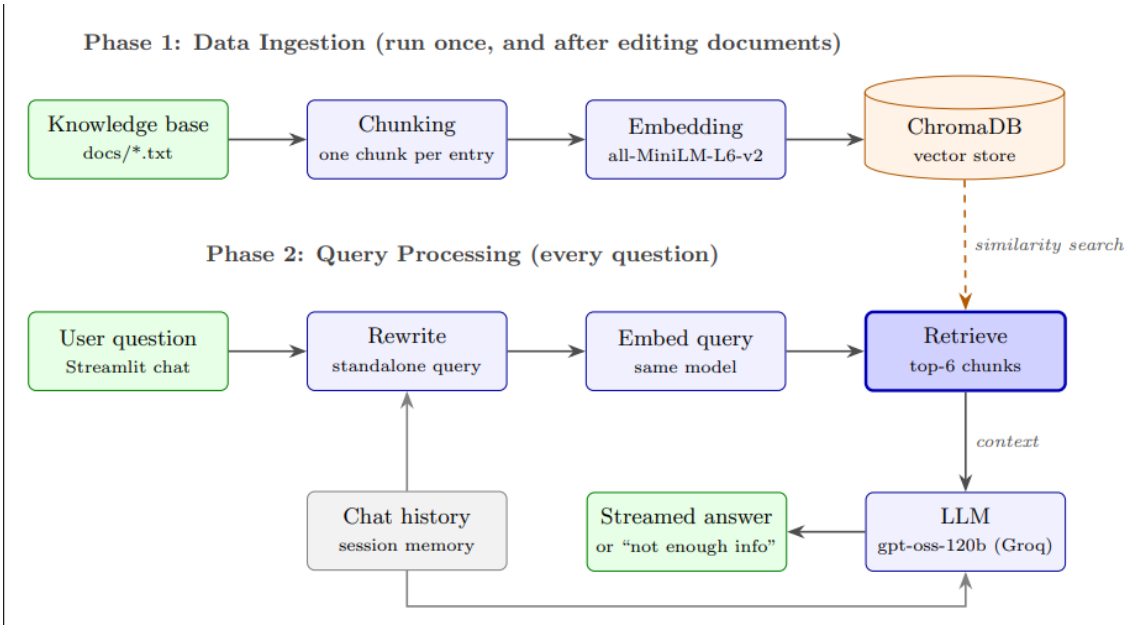
The architecture has two phases: Data Ingestion and Query Processing.

3.1 Data Ingestion Phase

The knowledge base is a set of .txt files covering Linux and Git commands, flags, how-to guides, errors and safety notes. Each topic is written as one "one chunk per entry" entry under category headings. The ingestion script splits each file into one chunk per entry, adds the category and heading to each chunk, converts it into a vector with an embedding model (all-MiniLM-L6-v2) and stores it in a ChromaDB vector database.

3.2 Query & Generation Phase

When the user asks a question, a follow-up is first rewritten into a standalone question using the chat history. The question is embedded with the same model, and ChromaDB returns the top 6 most similar chunks using cosine similarity. These chunks are placed in the prompt as context, and the LLM streams an answer based only on that context. If the context does not contain the answer, the model replies that it does not have enough information.



ARCHITECTURE DIAGRAM

4. Implementation Details & Tech Stack

Component	**Technology Used**	**Purpose**
Programming Language	Python 3	Core backend logic
Orchestration Framework	LangChain	Connecting LLMs, loaders and vector stores
Vector Database	ChromaDB (cosine similarity)	Storing and retrieving document embeddings
Embeddings Model	HuggingFace all-MiniLM-L6-v2	Converting text into vector representations
Large Language Model	gpt-oss-120b via Groq API	Generating answers from the retrieved context
User Interface	Streamlit	Chat UI with history, streaming and Troubleshoot mode

4.1 Sample Chunking Configuration

Fixed-size splitters can cut command entries in the middle. Instead, the documents are split at the “one chunk per entry” entry headings, so each chunk holds one complete topic, capped at 1200 characters:

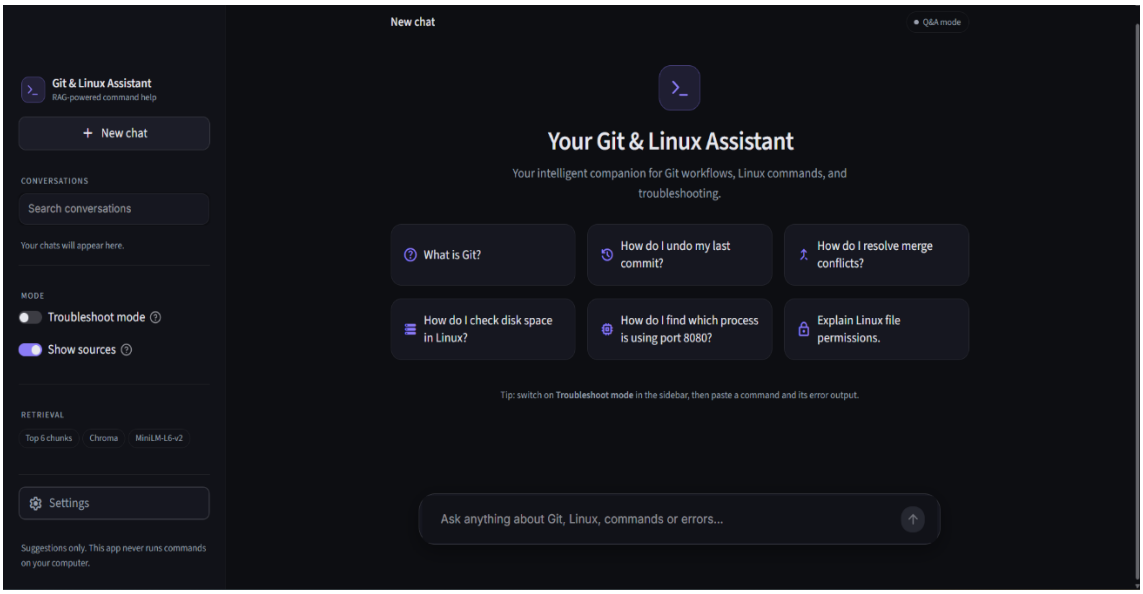
```
def split_documents (documents, max_chars=1200):
    # one chunk per entry; each piece repeats
    # The [category] and heading so it stays self-describing
    prefix = f"[{category}] \n{heading}\n"
    metadata = {"source": source, "category": category, "title":
title}
```

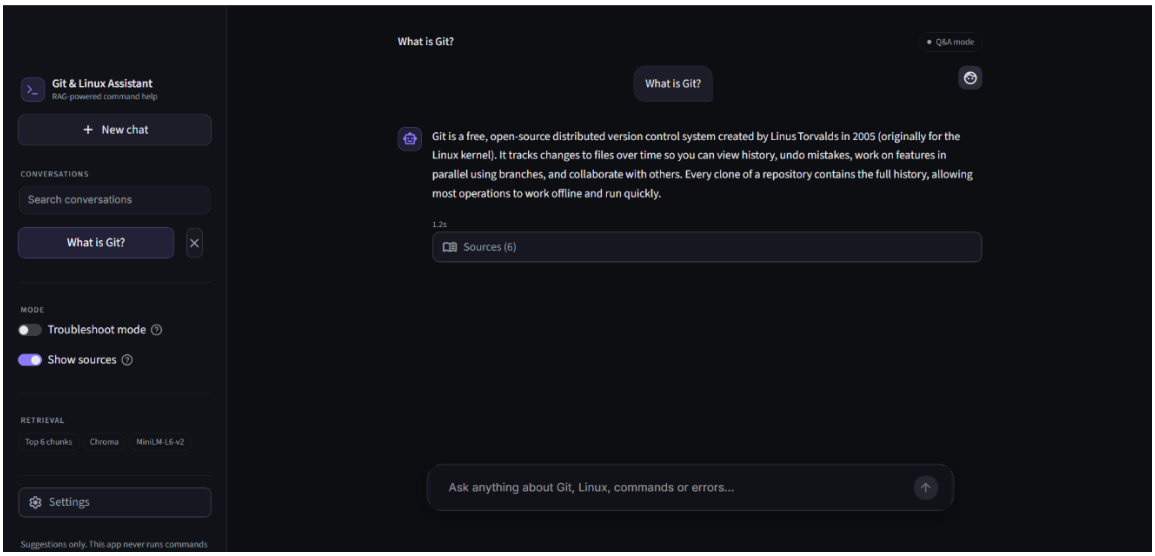
Retrieval settings: top-k = 6, temperature = 0, normalised embeddings. The index is rebuilt with python ingestion_pipeline.py --rebuild whenever the knowledge base changes.

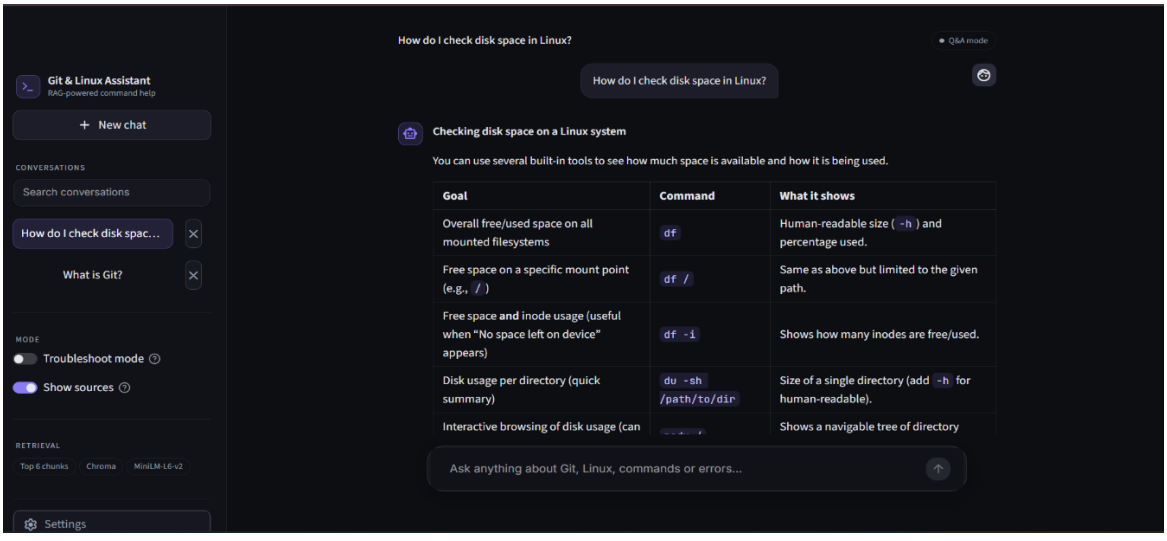
5. Results and Testing

The system was tested manually with sample questions covering Git concepts (“What is Git?”), recovery tasks (“How do I undo my last commit?”), flags (“What does -m mean in Git?”), Linux concepts (“What is Linux?”) and pasted terminal errors. Testing found and fixed these problems:

- **Stale index:** the database was not rebuilt after the documents changed. A --rebuild option fixed this.
- **Weak retrieval:** top-3 retrieval and generic chunking missed relevant entries. Entry-based chunking and top-6 retrieval improved this.
- **Short flag queries:** embeddings handle bare flags such as -m poorly. Plain-language flag entries were added and the question rewriter now preserves flags.
- **Missing concept entries:** “What is Linux?” was refused until concept entries were added.


New chat
Q&A mode
Git & Linux Assistant
RAG-powered command help
New chat
CONVERSATIONS
Search conversations
Your chats will appear here.
MODE
Troubleshoot mode
Show sources
RETRIEVAL
Top 6 chunks
Chroma
MiniLM-L6-v2
Settings
Suggestions only. This app never runs commands on your computer.
Your Git & Linux Assistant
Your intelligent companion for Git workflows, Linux commands, and troubleshooting.
What is Git?
How do I undo my last commit?
How do I resolve merge conflicts?
How do I check disk space in Linux?
How do I find which process is using port 8080?
Explain Linux file permissions.
Tip: switch on Troubleshoot mode in the sidebar, then paste a command and its error output.
Ask anything about Git, Linux, commands or errors...


What is Git?
Q&A mode
Git & Linux Assistant
RAG-powered command help
New chat
CONVERSATIONS
Search conversations
What is Git?
MODE
Troubleshoot mode
Show sources
RETRIEVAL
Top 6 chunks
Chroma
MiniLM-L6-v2
Settings
Suggestions only. This app never runs commands
What is Git?
Git is a free, open-source distributed version control system created by Linus Torvalds in 2005 (originally for the Linux kernel). It tracks changes to files over time so you can view history, undo mistakes, work on features in parallel using branches, and collaborate with others. Every clone of a repository contains the full history, allowing most operations to work offline and run quickly.
1.2s
Sources (6)
Ask anything about Git, Linux, commands or errors...


How do I check disk space in Linux?
Q&A mode
Git & Linux Assistant
RAG-powered command help
New chat
CONVERSATIONS
Search conversations
How do I check disk spac...
What is Git?
MODE
Troubleshoot mode
Show sources
RETRIEVAL
Top 6 chunks
Chroma
MiniLM-L6-v2
Settings
How do I check disk space in Linux?
Checking disk space on a Linux system
You can use several built-in tools to see how much space is available and how it is being used.
Goal
Command
What it shows
Overall free/used space on all mounted filesystems
df
Human-readable size (-h) and percentage used.
Free space on a specific mount point (e.g., /)
df /
Same as above but limited to the given path.
Free space and inode usage (useful when "No space left on device" appears)
df -i
Shows how many inodes are free/used.
Disk usage per directory (quick summary)
du -sh /path/to/dir
Size of a single directory (add -h for human-readable).
Interactive browsing of disk usage (can
Shows a navigable tree of directory
Ask anything about Git, Linux, commands or errors...

6. Conclusion & Future Scope

Conclusion: The Git & Linux Command Assistant combines semantic search over a curated knowledge base with a modern LLM to give grounded answers to command-line questions. Because answers come only from the retrieved entries, the assistant can refuse instead of guessing, and its knowledge can be extended by editing text files.

Future Scope: Planned work includes a formal evaluation set with measured accuracy, persistent chat history, hybrid (keyword and semantic) retrieval for flag queries, a larger knowledge base covering more tools such as Docker, SSH and shell scripting, and a feedback button to track which answers helped.